

NAT System Architecture

Quantitative Research Platform for Hyperliquid Perpetual Futures

NAT Project

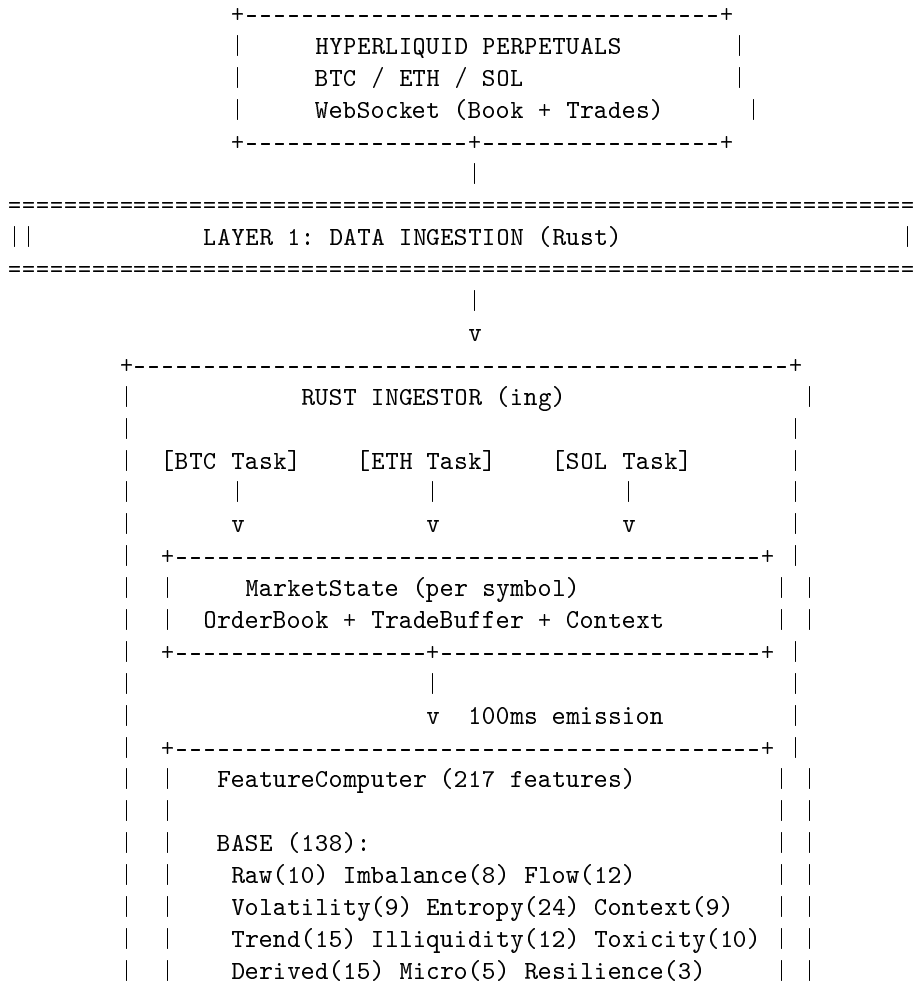
May 2026

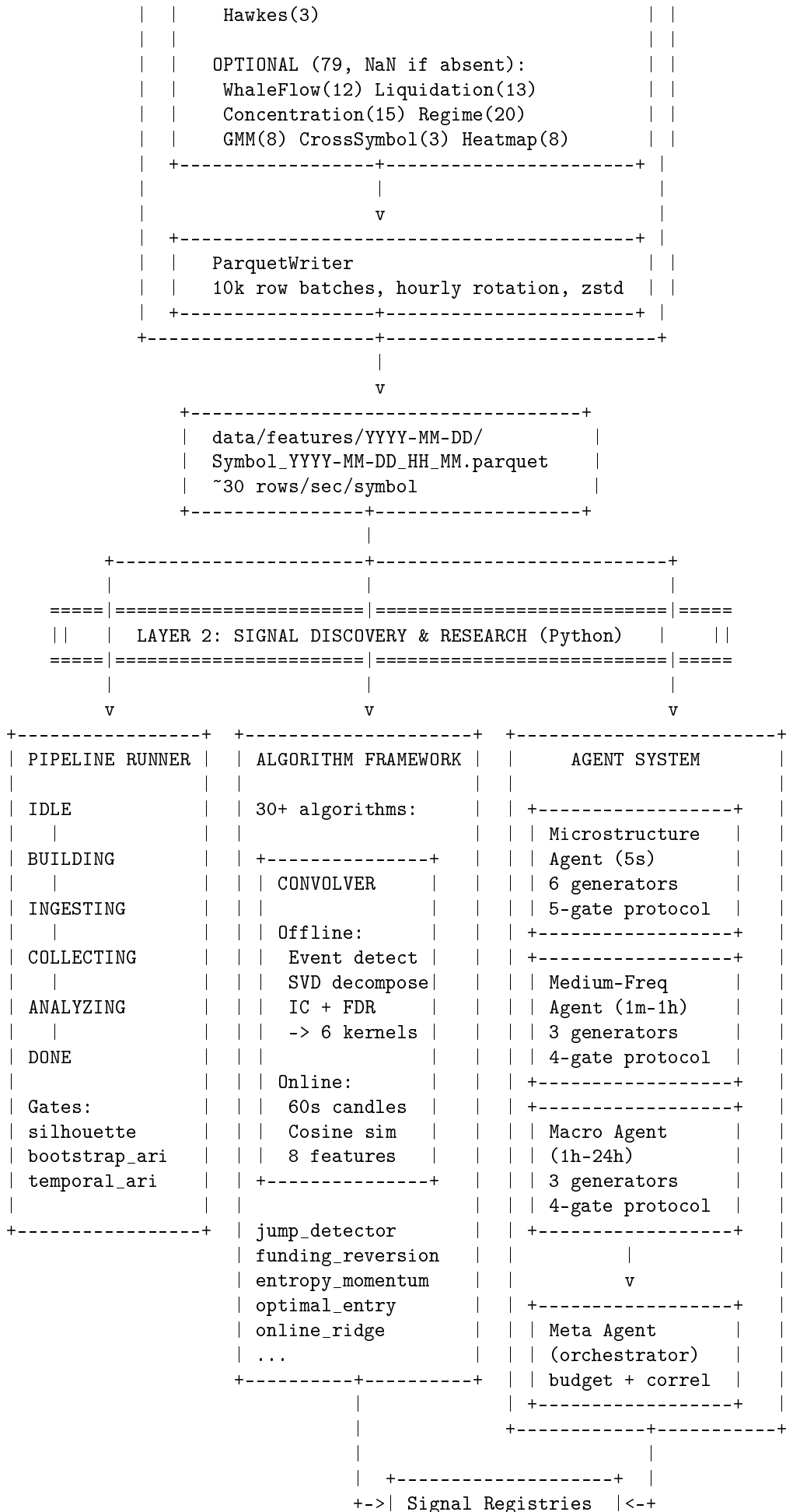
1 Overview

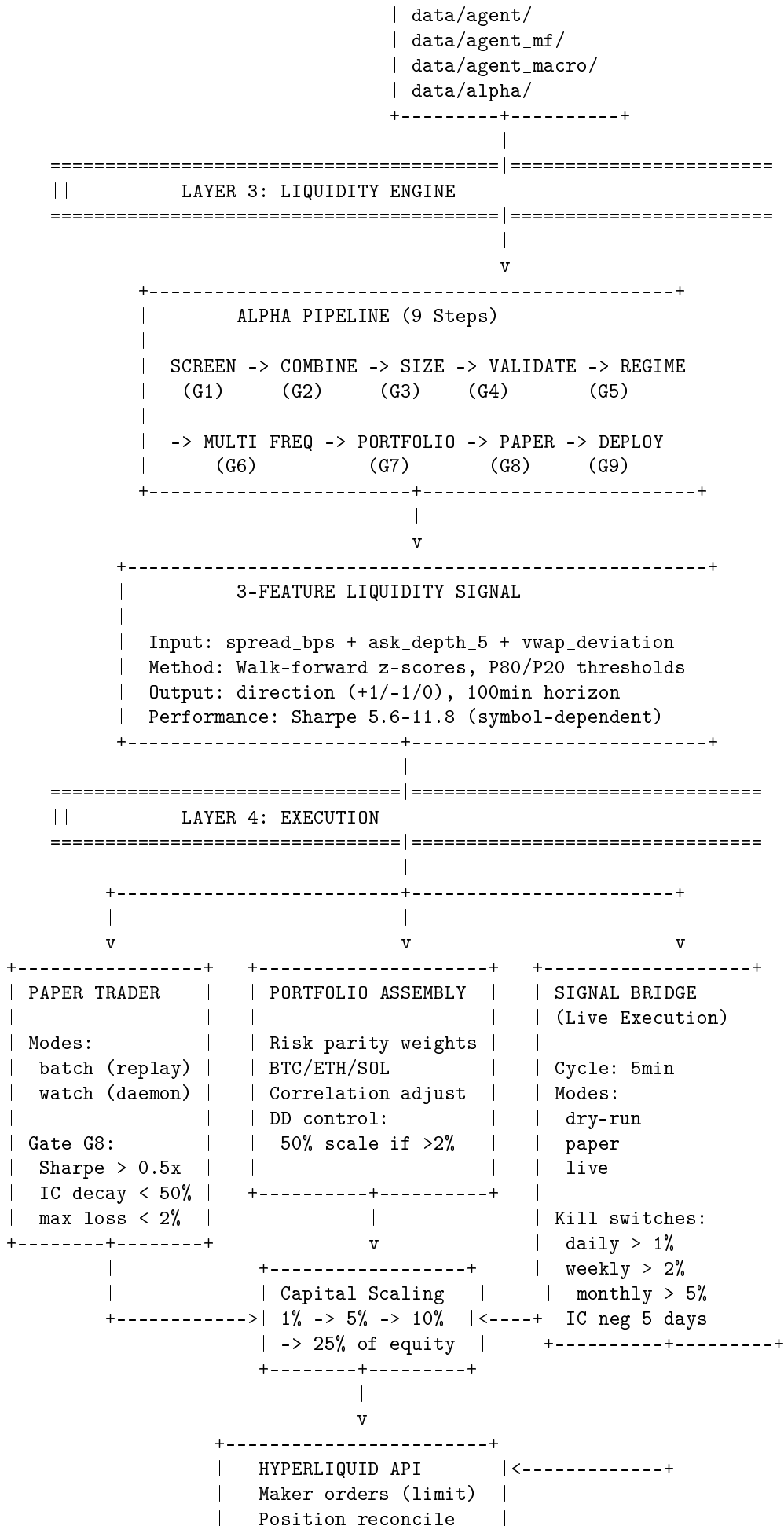
NAT is a quantitative research platform for extracting alpha signals from Hyperliquid perpetual futures. The system comprises four layers:

1. **Data Ingestion** — Rust real-time ingestor computing 217 features at 100ms intervals
2. **Signal Discovery** — Python-based research pipelines, algorithm framework, and autonomous agents
3. **Liquidity Engine** — Alpha pipeline with 9-step quality gates producing validated signals
4. **Execution** — Paper trading, portfolio assembly, and live order placement

2 System Architecture Diagram







+-----+

3 Layer 1: Data Ingestion (Rust)

The ingestor connects to Hyperliquid’s WebSocket API and computes 217 features per symbol at 100ms intervals.

3.1 Feature Categories

	Category	Count	Key Metrics
Base Features (138, always computed)			
Raw	10	Midprice, spread (bps), microprice, depth	
Imbalance	8	Qty/order/notional imbalance (L1/L5/L10)	
Flow	12	Trade count/volume, aggressor ratio, VWAP dev	
Volatility	9	Returns vol, Parkinson, Garman-Klass, z-score	
Entropy	24	Permutation, tick, vol-weighted tick entropy	
Context	9	Funding rate, OI, premium, 24h volume	
Trend	15	Momentum, R^2 , Hurst, EMA crossover	
Illiquidity	12	Kyle λ , Amihud, Hasbrouck, Roll spread	
Toxicity	10	VPIN, adverse selection, effective spread	
Derived	15	Entropy-trend interaction, regime type	
Microstructure	5	OBI dynamics, depth recovery, queue proxy	
Resilience	3	Order book resilience (state machine)	
Hawkes	3	Self-exciting trade intensity	
Optional Features (79, NaN-padded when absent)			
Whale Flow	12	Net flow, momentum, intensity, activity	
Liquidation	13	Cascade risk by price level, asymmetry	
Concentration	15	HHI, Gini, Theil, whale metrics	
Regime	20	Absorption, divergence, churn (minute-level)	
GMM	8	Regime label, probabilities, confidence	
Cross-Symbol	3	OBI divergence (multi-symbol)	
Heatmap	8	Spatial liquidation density, cascade chain	

3.2 Output Format

Parquet files organized as `data/features/YYYY-MM-DD/Symbol_HH_MM.parquet`. Row groups of 10,000 rows (~5.5 min for 3 symbols), zstd compressed, hourly rotation.

4 Layer 2: Signal Discovery

4.1 Convolver

Pattern-matching algorithm using data-driven SVD decomposition:

- **Offline:** Detect 6 event types (breakouts, turtle soup, traps) → extract $W=20$ candle windows → SVD on 4 channels (body, upper wick, lower wick, volume) → IC gate + BH-FDR correction ($q = 0.05$) → 6 surviving kernels
- **Online:** Accumulate ticks → form 60s candles → score sliding window against kernels via cosine similarity → 8 real-time features

4.2 Agent System

Three autonomous daemons operating at different timeframes:

Agent	Horizon	Generators	Gate Protocol
-------	---------	------------	---------------

Microstructure	5s	6	5-gate (IC, temporal, symbol, orthogonality, cost)
Medium-Freq	1m–1h	3	4-gate (IC, temporal, symbol, cost)
Macro	1h–24h	3	4-gate (IC, temporal, symbol, cost)

Meta Agent orchestrates cross-agent budget allocation, correlation dedup, and portfolio assembly.

4.3 Discovery Orchestrator

Continuous 6-hour cycles: DATA_HEALTH → SIGNAL_SWEEP → TRAINING → BACKTESTING → ALPHA_PIPELINE → REPORTING → SLEEPING.

5 Layer 3: Liquidity Engine

5.1 Alpha Pipeline (9 Steps)

Step	Name	Method	Gate
1	Screening	FDR on feature IC	G1: ≥ 5 significant features
2	Combining	IC-weighted, correlation dedup	G2: IC ratio $\geq 0.8 \times \max$
3	Sizing	Kelly-fraction, cost-aware	G3: 50% trade reduction
4	Validating	Walk-forward OOS	G4: Sharpe ≥ 0.5 , PF ≥ 1.2
5	Regime	GMM conditioning	G5: $1.5 \times$ IC ratio improvement
6	Multi-Freq	Timeframe ensemble	G6: composite $>$ individual
7	Portfolio	Risk parity, cross-asset	G7: DD $< 0.8 \times$ worst single
8	Paper	Live simulation	G8: Sharpe $> 0.5 \times$ backtest
9	Deploy	Model readiness	G9: all artifacts ready

5.2 3-Feature Liquidity Signal

- **Inputs:** raw_spread_bps, raw_ask_depth_5, flow_vwap_deviation
- **Method:** Walk-forward z-scores (train on 3 prior dates), composite = $\frac{1}{3}(z_{spread} + z_{depth} + z_{vwap})$
- **Entry:** Long when composite $\geq P_{80}$, short when $\leq P_{20}$
- **Exit:** Fixed 100-minute horizon (20 five-minute bars)
- **Performance:** Sharpe 5.6–11.8 (symbol-dependent), net 6.7–7.2 bps/trade

6 Layer 4: Execution

6.1 Paper Trading

Two modes: **batch** (replay all dates, compare to backtest) and **watch** (daemon monitoring for new data). Gate G8 requires paper Sharpe $> 0.5 \times$ backtest, IC decay $< 50\%$, max daily loss $< 2\%$.

6.2 Signal Bridge

Live execution daemon with 5-minute cycles:

- Reads latest bars → computes signal → reconciles position
- Modes: dry-run, paper, live (requires HL_PRIVATE_KEY)
- Kill switches: daily loss $> 1\%$, weekly DD $> 2\%$, monthly DD $> 5\%$, IC negative 5 days

6.3 Capital Scaling

Progressive deployment: 1% → 5% → 10% → 25% of equity. Portfolio-level drawdown control scales positions by 50% if DD exceeds 2%.

7 Component Interaction Matrix

Producer → Consumer	Data Format	Frequency
Hyperliquid → Ingestor	WebSocket JSON	~10ms
Ingestor → Parquet	Arrow batches (10k rows)	~5.5 min flush
Parquet → Algorithm Framework	DataFrame read	On-demand
Parquet → Agent Daemons	DataFrame read	1h / 2h / 4h cycles
Parquet → Liquidity Engine	5min bars	Walk-forward daily
Convolver Discovery → Kernels	.npz + .json	Offline (rare)
Convolver Online → Features	8 floats per candle	Every 60s
Alpha Pipeline → Registry	JSON	Per pipeline run
Agent → Registry	JSON	Per hypothesis pass
Registry → Paper Trader	Trade decisions	5min
Paper Trader → Signal Bridge	Validated signal	5min
Signal Bridge → Hyperliquid	REST API (orders)	5min

8 Key File Paths

Component	Entry Point
Rust Ingestor	rust/ing/src/main.rs
Feature Modules (26)	rust/ing/src/features/*.rs
Convolver (online)	scripts/algorithms/convolver.py
Convolver (discovery)	scripts/analysis/convolver_discovery.py
Pipeline Runner	scripts/pipeline_runner.py
Alpha Pipeline	scripts/alpha/alpha_pipeline.py
Liquidity Backtest	scripts/analysis/mf_liquidity_backtest.py
Paper Trader	scripts/alpha/paper_trader.py
Signal Bridge	scripts/execution/signal_bridge.py
Agent (micro)	scripts/agent/daemon.py
Agent (MF)	scripts/agent/mf_daemon.py
Agent (macro)	scripts/agent/macro_daemon.py
Meta Agent	scripts/agent/meta_daemon.py
Discovery Orchestrator	scripts/discovery_orchestrator.py